
SESSION-5

Task 1: To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 200
#define MAX_QUOTES 20

int main() {
    char input[MAX_INPUT];
    char quoted_strings[MAX_QUOTES][MAX_INPUT];
    int quote_count = 0;

    printf("Session 5 - Single Quote Parsing\n");
    printf("Enter a command with single quotes:\n");
    printf("myshell> ");

    fgets(input, sizeof(input), stdin);

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    int i = 0;

    /* Parse input and find single-quoted strings */
    while (input[i] != '\0') {
        if (input[i] == '\') {
            i++;

            int j = 0;

            /* Store everything until the closing quote */
            while (input[i] != '\0' && input[i] != '\') {
                if (j < MAX_INPUT - 1) {
                    quoted_strings[quote_count][j++] = input[i];
                }
                i++;
            }

            quoted_strings[quote_count][j] = '\0';
```

```

        /* Check for closing quote */
        if (input[i] == '\\') {
            if (quote_count < MAX_QUOTES - 1) {
                quote_count++;
            }
            i++;
        }
        else {
            printf("Error: Unclosed single quote.\n");
            return 1;
        }

    } else {
        i++;
    }
}

printf("\nQuoted strings found:\n");

if (quote_count == 0) {
    printf("No single-quoted strings found.\n");
}
else {
    for (int k = 0; k < quote_count; k++) {
        printf("Quoted String %d: [%s]\n",
            k + 1, quoted_strings[k]);
    }
}

printf("\nParsing validation:\n");

if (quote_count > 0) {
    printf("Single-quote parsing successful.\n");
}
else {
    printf("No quoted content to validate.\n");
}

return 0;
}

```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill3.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill3.1.c -o skill3.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill3.1
Session 5 - Single Quote Parsing
Enter a command with single quotes:
myshell> hello @world

Quoted strings found:
No single-quoted strings found.

Parsing validation:
No quoted content to validate.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Task 2: To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 300
#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 100

/* Expand environment variables such as $USER */
void expand_variable(char *input, char *output) {
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_INPUT - 1) {

        if (input[i] == '$') {
            i++;

            char variable[50];
            int k = 0;

            /* Read variable name */
            while (isalnum((unsigned char)input[i]) ||
                input[i] == '_') {

                if (k < 49) {
                    variable[k++] = input[i];
                }

                i++;
            }
        }
    }
}
```

```

        variable[k] = '\0';

        /* Get environment variable value */
        char *value = getenv(variable);

        if (value != NULL) {
            int v = 0;

            while (value[v] != '\0' &&
                   j < MAX_INPUT - 1) {
                output[j++] = value[v++];
            }
        } else {
            output[j++] = input[i++];
        }
    }

    output[j] = '\0';
}

int main() {

    char input[MAX_INPUT];
    char expanded[MAX_INPUT];

    char tokens[MAX_TOKENS][MAX_TOKEN_LENGTH];

    int token_count = 0;
    int i = 0;

    printf("Session 5 - Double Quote Parsing\n");
    printf("Double quotes preserve spaces and allow variable expansion.\n");
    printf("Enter a command:\n");
    printf("myshell> ");

    fgets(input, sizeof(input), stdin);

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    /* Handle empty command */
    if (strlen(input) == 0) {
        printf("Empty command.\n");
        return 0;
    }
}

```

```

/* Parse input */
while (input[i] != '\0' &&
      token_count < MAX_TOKENS) {

    /* Skip spaces */
    while (isspace((unsigned char)input[i])) {
        i++;
    }

    if (input[i] == '\0') {
        break;
    }

    int j = 0;

    /* Double-quoted string */
    if (input[i] == '"') {

        i++;

        while (input[i] != '\0' &&
              input[i] != '"') {

            if (j < MAX_TOKEN_LENGTH - 1) {
                tokens[token_count][j++] = input[i];
            }

            i++;
        }

        /* Check closing quote */
        if (input[i] == '"') {
            i++;
        } else {
            printf("Error: Unclosed double quote.\n");
            return 1;
        }

        tokens[token_count][j] = '\0';
        token_count++;
    }

    /* Normal token */
    else {

        while (input[i] != '\0' &&
              !isspace((unsigned char)input[i]) &&

```

```

        input[i] != '"') {
            if (j < MAX_TOKEN_LENGTH - 1) {
                tokens[token_count][j++] = input[i];
            }
            i++;
        }

        tokens[token_count][j] = '\0';

        if (j > 0) {
            token_count++;
        }
    }

    /* Display tokens before expansion */
    printf("\nTokens before variable expansion:\n");

    for (int k = 0; k < token_count; k++) {
        printf("Token %d: [%s]\n", k + 1, tokens[k]);
    }

    /* Variable expansion */
    printf("\nVariable expansion:\n");

    for (int k = 0; k < token_count; k++) {
        expand_variable(tokens[k], expanded);

        printf("Token %d: [%s]\n",
            k + 1, expanded);
    }

    /* Validation */
    printf("\nParsing validation:\n");
    printf("Double-quote parsing successful.\n");
    printf("Spaces inside quotes were preserved.\n");
    printf("Variable expansion was processed.\n");

    return 0;
}

```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill3.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill3.2.c -o skill3.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill3.2
Session 5 - Double Quote Parsing
Double quotes preserve spaces and allow variable expansion.
Enter a command:
myshell> hello world

Tokens before variable expansion:
Token 1: [hello]
Token 2: [world]

Variable expansion:
Token 1: [hello]
Token 2: [world]

Parsing validation:
Double-quote parsing successful.
Spaces inside quotes were preserved.
Variable expansion was processed.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

SESSION-6

Task 1: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

CODE:

GNU nano 8.7.1

```
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 300

int main() {
    char input[MAX_INPUT];
    char output[MAX_INPUT];

    int i = 0;
    int j = 0;
    int escaped = 0;

    printf("Session 6 - Escape Sequence Parsing\n");
    printf("Enter a command with escaped spaces or special characters:\n");
    printf("myshell> ");

    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    /* Handle empty input */
    if (strlen(input) == 0) {
        printf("Empty input.\n");
        return 0;
    }

    /* Process escape sequences */
    while (input[i] != '\0' && j < MAX_INPUT - 1) {

        if (escaped) {
            output[j++] = input[i];
            escaped = 0;
        }
        else if (input[i] == '\\') {
            escaped = 1;
        }
        else {
            output[j++] = input[i];
        }
    }
}
```

```

        i++;
    }

    /* Check if input ends with escape character */
    if (escaped) {
        printf("Warning: Escape character at the end of input.\n");
    }

    output[j] = '\\0';

    printf("\nOriginal input:\n");
    printf("%s\n", input);

    printf("\nProcessed input:\n");
    printf("%s\n", output);

    printf("\nParser validation:\n");
    printf("Escape sequences processed successfully.\n");
    printf("Escaped characters were preserved.\n");

    return 0;
}

```

OUTPUT:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill3.3.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill3.3.c -o skill3.3
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill3.3
Session 6 - Escape Sequence Parsing
Enter a command with escaped spaces or special characters:
myshell> ls

Original input:
ls

Processed input:
ls

Parser validation:
Escape sequences processed successfully.
Escaped characters were preserved.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

Task 2: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;
    int status;

    printf("Session 6 - Process Execution\n");

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        /* Child process */
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("Executing: ls -l\n");

        execlp("ls", "ls", "-l", NULL);

        /* This runs only if exec fails */
        perror("exec failed");
        exit(EXIT_FAILURE);
    }
    else {
        /* Parent process */
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        printf("Parent waiting for child...\n");
    }
}
```

```
waitpid(pid, &status, 0);

if (WIFEXITED(status)) {
    printf("\nChild exited with status: %d\n",
        WEXITSTATUS(status));
}
else {
    printf("\nChild terminated abnormally.\n");
}

printf("Parent process completed.\n");
}

return 0;
}
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID885B3:~$ nano skill3.4.c
vishwa_teja_veerandi@LAPTOP-DID885B3:~$ gcc skill3.4.c -o skill3.4
vishwa_teja_veerandi@LAPTOP-DID885B3:~$ ./skill3.4
Session 6 - Process Execution

Parent Process
Parent PID: 584
Child PID: 585
Parent waiting for child...

Child Process
Child PID: 585
Parent PID: 584
Executing: ls -l
total 476
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 0 Aug 8 04:16 File2.txt
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 107 Aug 20 08:15 Makefile
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 12 Aug 18 07:08 destination.txt
drwxr-xr-x 2 vishwa_teja_veerandi vishwa_teja_veerandi 4096 Aug 1 04:16 linux
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 15952 Jul 28 08:04 myprogram
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 16216 Aug 18 07:08 practical2.1
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 1232 Aug 18 06:58 practical2.1.c
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 0 Aug 18 07:00 practical2.1.txt
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 16296 Aug 18 06:34 practical3.1
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 1069 Aug 11 06:28 practical3.1.c
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 16088 Aug 18 06:36 practical3.2
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 357 Aug 11 06:32 practical3.2.c
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 16392 Aug 18 06:43 problem1
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 1116 Aug 1 02:56 problem1.c
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 0 Aug 1 04:17 problem1.txt
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 25 Aug 20 06:53 sample.txt
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 14 Aug 18 07:49 sample2.txt
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 15952 Aug 20 08:06 skill1.1
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 140 Aug 20 08:04 skill1.1.c
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 16296 Aug 20 08:27 skill1.2
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 730 Aug 20 08:25 skill1.2.c
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 16096 Aug 20 08:36 skill1.3
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 412 Aug 20 08:36 skill1.3.c
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 16080 Aug 20 08:18 skill2
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 16664 Aug 20 09:44 skill2.1
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 4227 Aug 20 09:37 skill2.1.c
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 16384 Aug 20 10:03 skill2.2
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 2188 Aug 20 10:02 skill2.2.c
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 16184 Aug 20 10:06 skill2.3
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 2233 Aug 20 10:06 skill2.3.c
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 16264 Aug 20 10:09 skill2.4
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 2413 Aug 20 10:09 skill2.4.c
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 341 Aug 8 03:34 skill2.c
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 16144 Aug 8 04:09 skill3
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 16176 Aug 20 10:17 skill3.1
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 1842 Aug 20 10:17 skill3.1.c
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 16312 Aug 20 10:21 skill3.2
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 3768 Aug 20 10:21 skill3.2.c
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 16176 Aug 21 04:37 skill3.3
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 1337 Aug 21 04:37 skill3.3.c
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 16352 Aug 21 04:39 skill3.4
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 1198 Aug 21 04:39 skill3.4.c
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 377 Aug 8 03:37 skill3.c
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 16176 Aug 8 04:15 skill3main2
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 478 Aug 8 03:39 skill3main2.c
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 15 Aug 8 04:12 skill3main2.txt
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 1 Aug 11 07:09 skill4.1.c
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 12 Aug 19 09:56 source.txt
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 16216 Aug 20 08:30 sum1
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 498 Aug 4 06:58 sum1.c
-rwxr-xr-x 1 vishwa_teja_veerandi vishwa_teja_veerandi 16472 Aug 20 08:20 task2
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 1122 Aug 8 03:05 task2.c
drwxr-xr-x 2 vishwa_teja_veerandi vishwa_teja_veerandi 4096 Aug 1 04:13 teja
drwxr-xr-x 2 vishwa_teja_veerandi vishwa_teja_veerandi 4096 Jul 25 04:08 vishwa
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 81 Jul 25 04:10 vishwa.c

Child exited with status: 0
Parent process completed.
vishwa_teja_veerandi@LAPTOP-DID885B3:~$ |
```